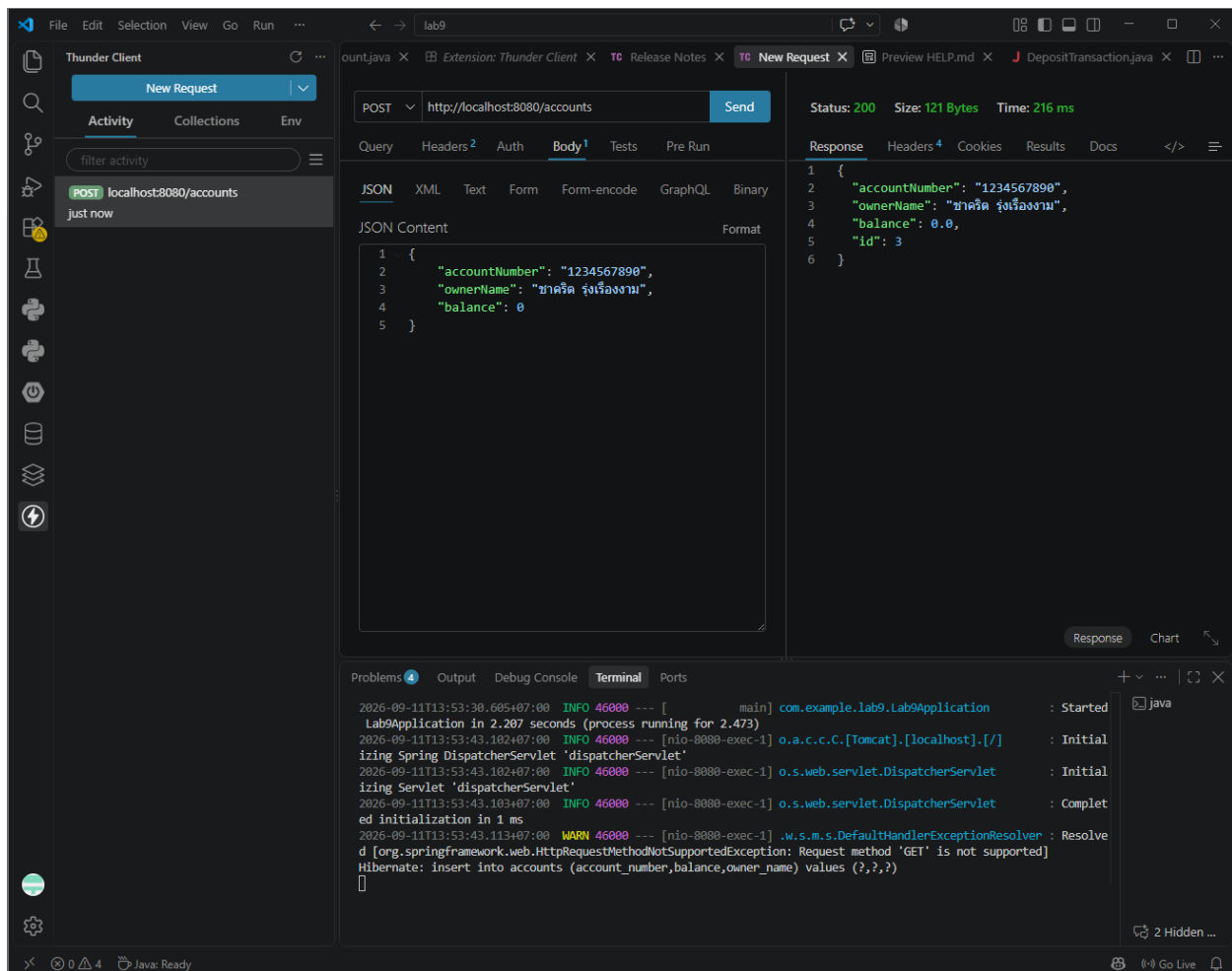


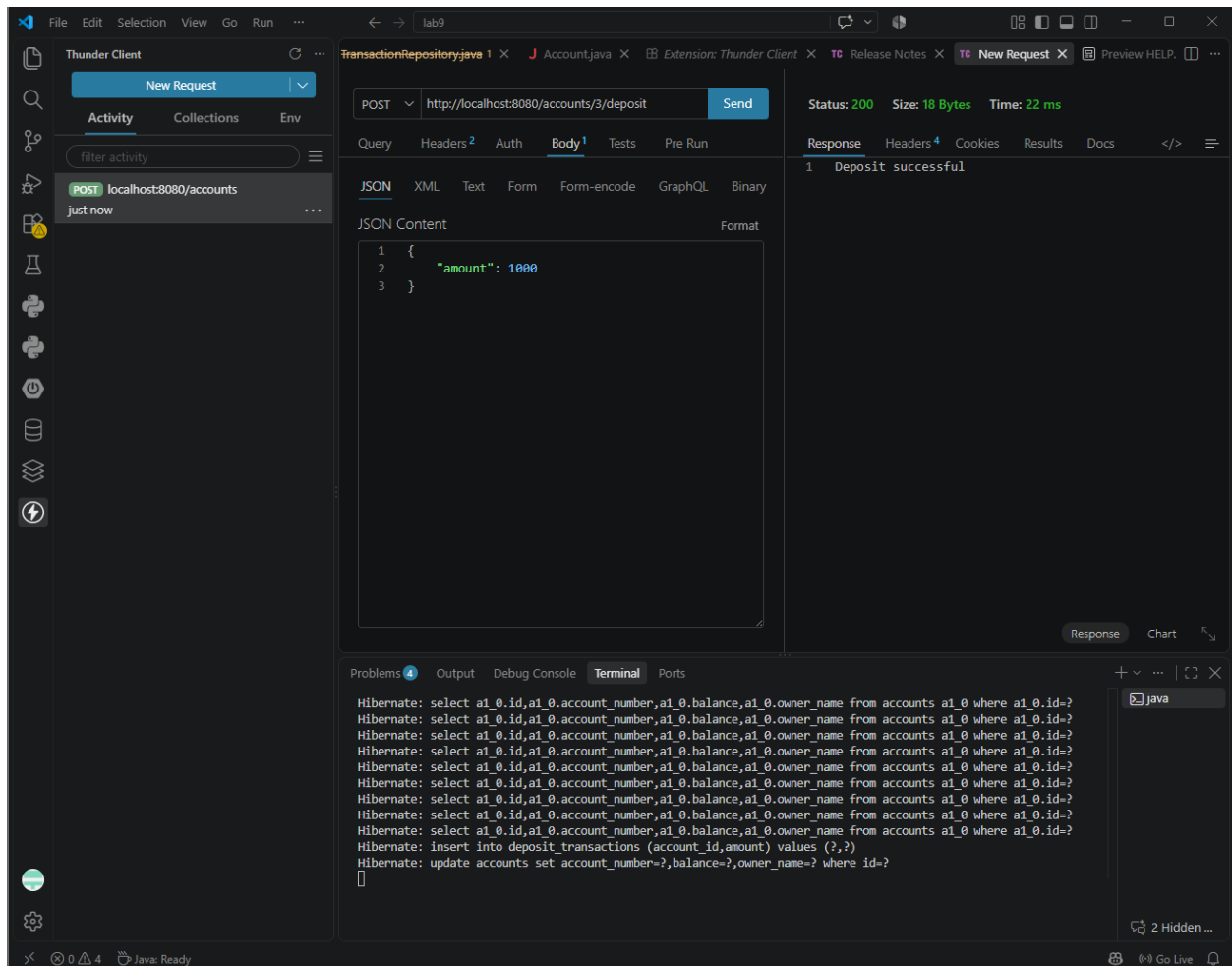


1. การสร้าง Account

ระบบได้รับ HTTP Request แบบ POST เข้ามาที่ Endpoint /accounts พร้อมข้อมูล JSON ของเจ้าของบัญชี ชื่อ "ชาคริต รุ่งเรืองงาม" เลขที่บัญชี "1234567890" และยอดเงินเริ่มต้น 0

- AccountController ได้ส่งต่อข้อมูลไปยัง AccountService.createAccount() เพื่อทำการบันทึกข้อมูลลงฐานข้อมูลผ่าน AccountRepository
- ในแถบ Terminal ด้านล่างแสดง Log คำสั่ง SQL จาก Hibernate: Hibernate: insert into accounts (account_number,balance,owner_name) values (?,?,?) แสดงว่ามีการเพิ่มข้อมูลลงในตาราง accounts จริงในฐานข้อมูล
- ระบบตอบกลับด้วย HTTP Status 200 OK พร้อมข้อมูล Account ที่ถูกบันทึกสำเร็จ ซึ่งมีการเจนรหัส Primary Key (id) ให้อัตโนมัติเป็น 3

สรุปผล: การสร้าง Account ใหม่ผ่าน API POST /accounts ทำงานได้อย่างถูกต้อง ข้อมูลถูกบันทึกลงฐานข้อมูลอย่างสมบูรณ์ และได้ Response ข้อมูลพร้อม ID กลับมาตรงตามโครงสร้างที่กำหนด



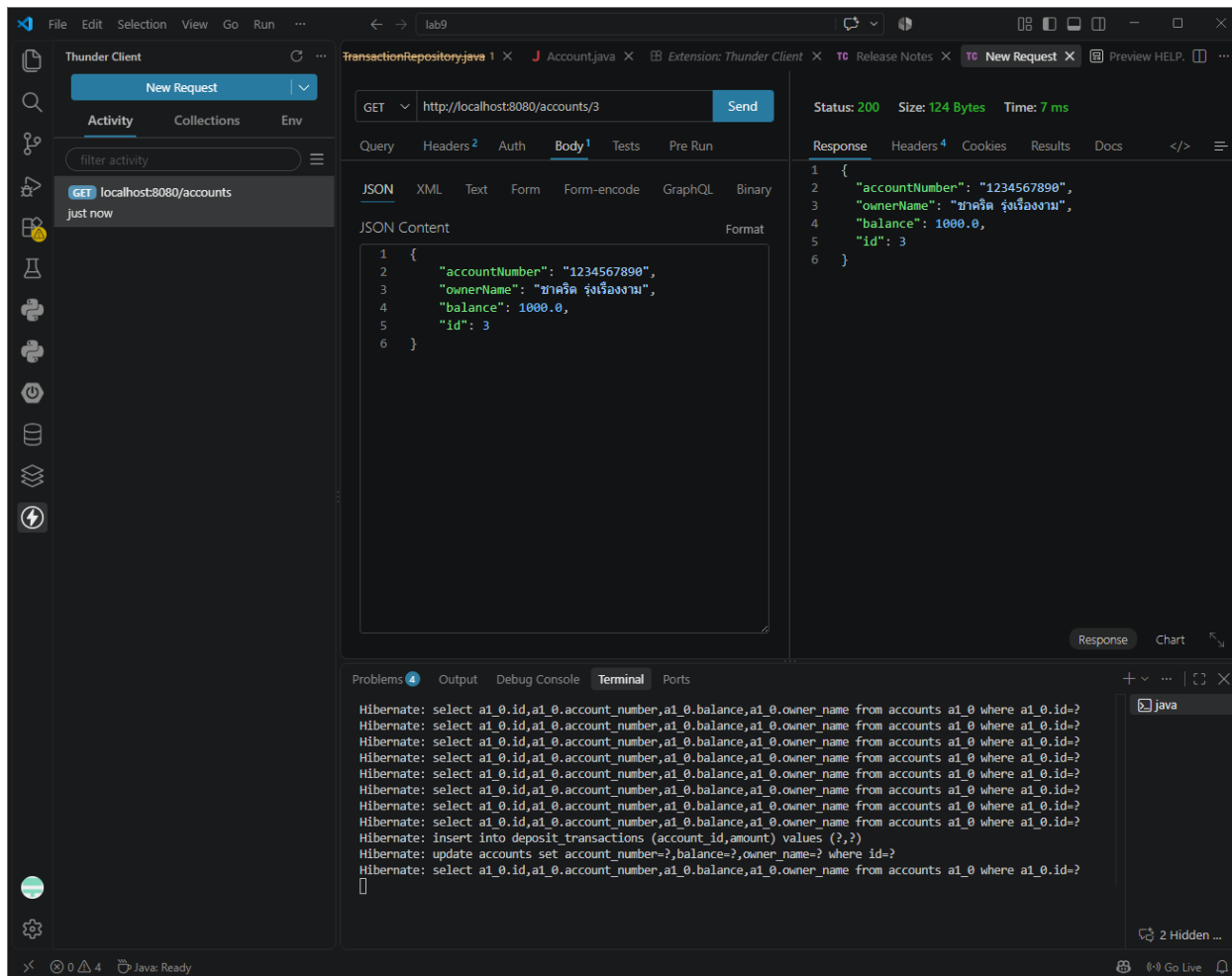
2. การฝากเงินสำเร็จ

ระบบได้รับ HTTP Request แบบ POST เข้ามาที่ Endpoint /accounts พร้อมข้อมูล JSON ของเจ้าของบัญชีชื่อ "ชาคริต รุ่งเรืองงาม" เลขที่บัญชี "1234567890" และยอดเงินเริ่มต้น 0

- AccountController ได้ส่งต่อข้อมูลไปยัง AccountService.createAccount() เพื่อทำการบันทึกข้อมูลลงฐานข้อมูลผ่าน AccountRepository
- ในแถบ Terminal ด้านล่างแสดง Log คำสั่ง SQL จาก Hibernate: Hibernate: insert into accounts (account_number,balance,owner_name) values (?,?,:) แสดงว่ามีการเพิ่มข้อมูลลงในตาราง accounts จริงในฐานข้อมูล

- ระบบตอบกลับด้วย HTTP Status 200 OK พร้อมข้อมูล Account ที่ถูกบันทึกสำเร็จ ซึ่งมีการเจนรหัส Primary Key (id) ให้อัตโนมัติเป็น 3

สรุปผล: การสร้าง Account ใหม่ผ่าน API POST /accounts ทำงานได้อย่างถูกต้อง ข้อมูลถูกบันทึกลงฐานข้อมูลอย่างสมบูรณ์ และได้ Response ข้อมูลพร้อม ID กลับมาตรงตามโครงสร้างที่กำหนด

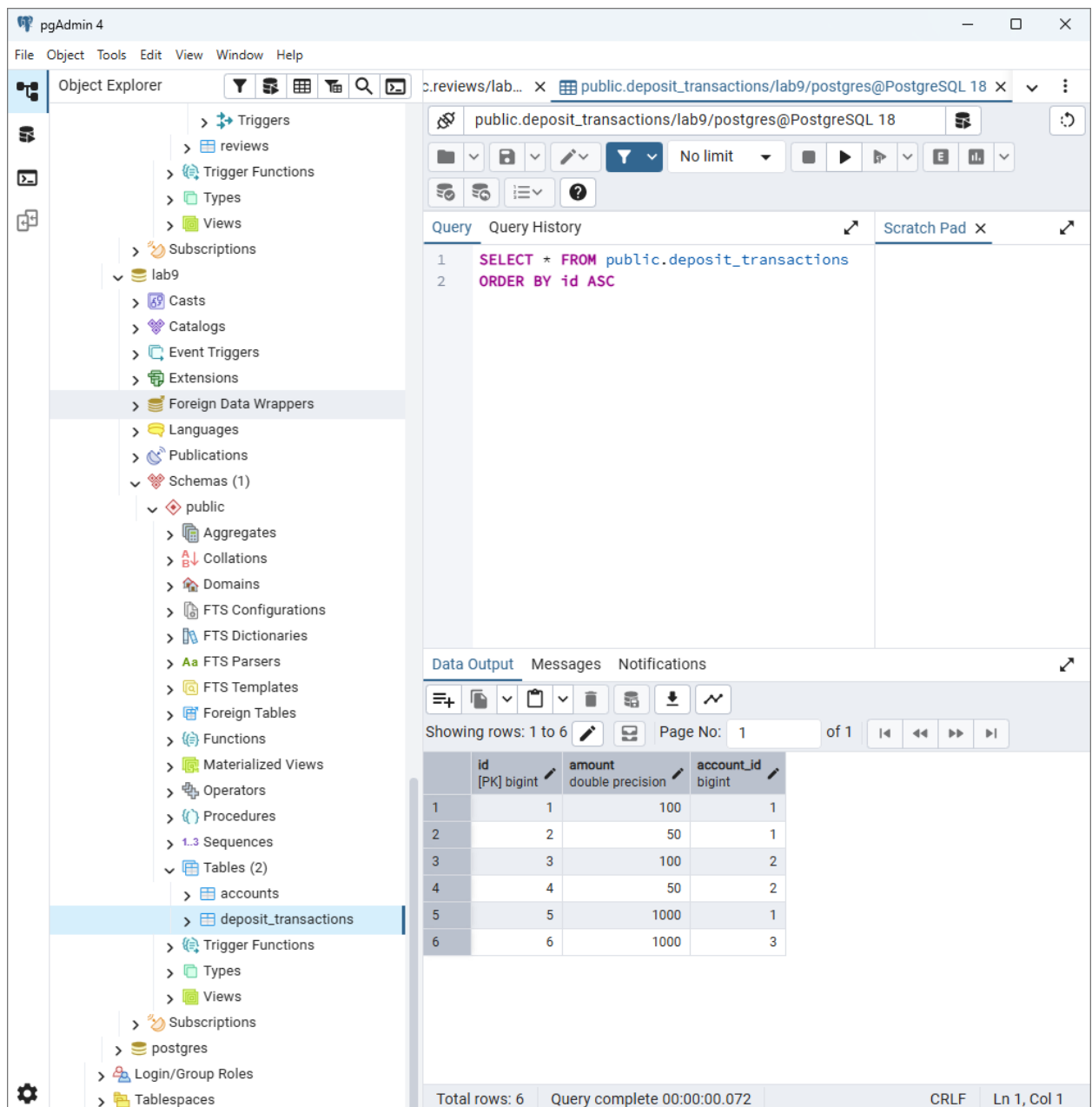


3. ตรวจสอบ Account

ระบบได้รับ HTTP Request แบบ GET เข้ามาที่ Endpoint /accounts/3 เพื่อร้องขอข้อมูลบัญชีหมายเลข ID 3

- AccountController ได้เรียกใช้งาน AccountService.getAccountById(3) เพื่ออ่านข้อมูลจากตาราง accounts

- ในแถบ Terminal แสดง Log คำสั่ง SQL จาก Hibernate ล่าสุด: Hibernate: select a1_0.id,a1_0.account_number,a1_0.balance,a1_0.owner_name from accounts a1_0 where a1_0.id=?
- ผลลัพธ์แสดงยอดเงินคงเหลือ (balance) เปลี่ยนจาก 0.0 เพิ่มขึ้นเป็น 1000.0 อย่างถูกต้อง ซึ่งเป็นผลมาจากการทำการรายการฝากเงินผ่าน @Transactional ในขั้นตอนก่อนหน้านี้
สรุปผล: การเรียกดูข้อมูลบัญชีผ่าน API GET /accounts/{id} ทำงานได้ถูกต้อง ข้อมูลที่แสดงยืนยันว่าการฝากเงินจำนวน 1000 บาทได้ถูก COMMIT บันทึกลงฐานข้อมูลจริงอย่างสมบูรณ์



The screenshot shows the pgAdmin 4 interface. On the left, the Object Explorer displays the database structure, with the 'public' schema selected. The 'deposit_transactions' table is highlighted. The main pane shows a SQL query:

```
SELECT * FROM public.deposit_transactions
ORDER BY id ASC
```

The query results are displayed in a table with 6 rows and 4 columns: id, amount, account_id, and account_name. The status bar at the bottom indicates 'Total rows: 6' and 'Query complete 00:00:00.072'.

id	amount	account_id	account_name
1	100	1	Account 1
2	50	1	Account 1
3	100	2	Account 2
4	50	2	Account 2
5	1000	1	Account 1
6	1000	3	Account 3

4. ตรวจสอบ DepositTransaction

- การตรวจสอบข้อมูลในตาราง deposit_transactions ผ่านโปรแกรม pgAdmin 4 พบรายการฝากเงินล่าสุดลำดับที่ **id = 6**
- ข้อมูลประกอบด้วยจำนวนเงินฝาก **amount = 1000** และ Foreign Key อ้างอิงไปยังบัญชี **account_id = 3**
- ข้อมูลแถวที่ 6 นี้ตรงกับรายการฝากเงินจำนวน 1000 บาท เข้าบัญชีหมายเลข ID 3 ที่ได้ทำรายการผ่าน API POST /accounts/3/deposit ในขั้นตอนก่อนหน้า
- เป็นการยืนยันว่ากระบวนการใน DepositService.deposit() ที่รันภายใต้ @Transactional ได้ทำการบันทึกข้อมูลประวัติการฝากเงิน (DepositTransaction) ลงในฐานข้อมูล PostgreSQL จริงอย่างสมบูรณ์

สรุปผล: ข้อมูลประวัติการฝากเงินถูกบันทึกลงในตาราง deposit_transactions อย่างถูกต้องสอดคล้องกับการเพิ่มขึ้นของยอดเงินในตาราง accounts ยืนยันการทำงานร่วมกันของ Database Transaction ที่สมบูรณ์

Thunder Client

New Request

Activity Collections Env

filter activity

POST localhost:8080/accounts/3/deposit 1 min ago

GET localhost:8080/accounts just now

POST http://localhost:8080/accounts/3/deposit Send

Status: 500 Size: 114 Bytes Time: 483 ms

Query Headers 2 Auth Body 1 Tests Pre Run

JSON XML Text Form Form-encode GraphQL Binary

JSON Content

```
1 {
2   "amount": 1000
3 }
```

Response Headers 4 Cookies Results Docs </>

```
1 {
2   "timestamp": "2026-09-11T07:07:48.573Z",
3   "status": 500,
4   "error": "Internal Server Error",
5   "path": "/accounts/3/deposit"
6 }
```

Problems 4 Output Debug Console Terminal Ports

```
0.24.jar:11.0.24]
    at org.apache.tomcat.util.threads.ThreadPoolExecutor.runWorker(ThreadPoolExecutor.java:949) ~[tomcat-embed-core-11.0.24.jar:11.0.24]
    at org.apache.tomcat.util.threads.ThreadPoolExecutor$Worker.run(ThreadPoolExecutor.java:483) ~[tomcat-embed-core-11.0.24.jar:11.0.24]
    at org.apache.tomcat.util.threads.TaskThread$WrappingRunnable.run(TaskThread.java:74) ~[tomcat-embed-core-11.0.24.jar:11.0.24]
    at java.base/java.lang.Thread.run(Thread.java:1447) ~[na:na]

Hibernate: select a1_0.id,a1_0.account_number,a1_0.balance,a1_0.owner_name from accounts a1_0 where a1_0.id=?
Hibernate: select a1_0.id,a1_0.account_number,a1_0.balance,a1_0.owner_name from accounts a1_0 where a1_0.id=?
[]
```

java

2 Hidden ...

Java: Ready

Go Live

The screenshot shows the pgAdmin 4 interface. On the left, the Object Explorer displays the database structure, with the 'public' schema selected. The 'deposit_transactions' table is highlighted. The main pane shows a SQL query:

```
SELECT * FROM public.deposit_transactions
ORDER BY id ASC
```

The query results are displayed in a table with 6 rows. The columns are 'id' (PK), 'amount', and 'account_id'.

id [PK] bigint	amount double precision	account_id bigint
1	100	1
2	50	1
3	100	2
4	50	2
5	1000	1
6	1000	3

The status bar at the bottom indicates 'Total rows: 6', 'Query complete 00:00:00.072', and 'CRLF Ln 1, Col 1'.

5. ทดลอง Rollback

เมื่อเกิด RuntimeException ขึ้นระหว่างกระบวนการฝากเงิน Annotation @Transactional ที่ครอบเมธอด deposit() ไว้ จะตรวจจับข้อผิดพลาดและสั่งยกเลิกรายการทั้งหมดทันที

- การเปลี่ยนแปลงข้อมูลที่เกิดขึ้นก่อนหน้าการเกิด Error (เช่น การคำนวณหรือเตรียมอัปเดตยอดเงินคงเหลือ) จะถูก **ROLLBACK** กลับคืนสู่สถานะเดิมทั้งหมดก่อนเริ่มทำรายการ

- ในรูปภาพจาก Thunder Client แสดงผลลัพธ์เป็น 500 Internal Server Error และเมื่อตรวจสอบใน pgAdmin 4 ตาราง deposit_transactions ก็ไม่มีการเพิ่มแถวข้อมูลใหม่ขึ้นมาแต่อย่างใด

สรุปผล: การทำงานของ @Transactional ช่วยป้องกันปัญหาข้อมูลขัดแย้งได้อย่างสมบูรณ์ โดยหากขั้นตอนใดขั้นตอนหนึ่งของ Transaction เกิด Error ระบบจะทำการ **Rollback** การเปลี่ยนแปลงทั้งหมดคืนกลับเป็นสภาพเดิมทันที ทำให้มั่นใจได้ว่าข้อมูลในฐานข้อมูลจะมีความถูกต้องตามหลัก **Atomicity (All or Nothing)**

Thunder Client

New Request

Activity Collections Env

filter activity

POST localhost:8080/accounts/3/deposit just now

GET localhost:8080/accounts just now

GET http://localhost:8080/accounts/3 Send

Status: 200 Size: 124 Bytes Time: 21 ms

Response Headers Cookies Results Docs </>

```
1 {
2   "accountNumber": "1234567890",
3   "ownerName": "ชาคริต ینگเรืองงาม",
4   "balance": 2000.0,
5   "id": 3
6 }
```

JSON Content Format

```
1 {
2   "accountNumber": "1234567890",
3   "ownerName": "ชาคริต ینگเรืองงาม",
4   "balance": 1000.0,
5   "id": 3
6 }
```

Problems 5 Output Debug Console Terminal Ports

```
at org.apache.tomcat.util.net.SocketProcessorBase.run(SocketProcessorBase.java:74) ~[tomcat-embed-core-11.0.24.jar:11.0.24]
at org.apache.tomcat.util.threads.ThreadPoolExecutor.runWorker(ThreadPoolExecutor.java:949) ~[tomcat-embed-core-11.0.24.jar:11.0.24]
at org.apache.tomcat.util.threads.ThreadPoolExecutor$Worker.run(ThreadPoolExecutor.java:483) ~[tomcat-embed-core-11.0.24.jar:11.0.24]
at org.apache.tomcat.util.threads.TaskThread$WrappingRunnable.run(TaskThread.java:74) ~[tomcat-embed-core-11.0.24.jar:11.0.24]
at java.base/java.lang.Thread.run(Thread.java:1447) ~[na:na]

Hibernate: select a1_0.id,a1_0.account_number,a1_0.balance,a1_0.owner_name from accounts a1_0 where a1_0.id=?
[]
```

Java

2 Hidden ...

Java: Ready

Go Live

The screenshot shows the pgAdmin 4 interface. On the left, the Object Explorer shows the database structure, with the 'public' schema selected. The 'deposit_transactions' table is highlighted. In the center, the Query Editor shows the following SQL query:

```
SELECT * FROM public.deposit_transactions
ORDER BY id ASC
```

Below the query editor, the Data Output tab shows the results of the query. The results are displayed in a table with 7 rows and 3 columns: id, amount, and account_id.

id	amount	account_id
1	100	1
2	50	1
3	100	2
4	50	2
5	1000	1
6	1000	3
7	1000	3

At the bottom of the Data Output tab, it shows 'Total rows: 7' and 'Query complete 00:00:00.217'.

6. เปรียบเทียบมี @Transactional กับไม่มี @Transactional

อธิบายผลการทดลองโดยละเอียด

1. กรณีมี @Transactional (ทำการ Rollback ได้สำเร็จ)

- พฤติกรรม: เมธอด deposit() ถูกมองเป็นงานก้อนเดียวกันทั้งหมด (All or Nothing)

- **ผลลัพธ์:** เมื่อเกิด Exception/Error ขึ้นในขั้นตอนใดขั้นตอนหนึ่ง ระบบจะทำการยกเลิกคำสั่งที่ทำไปแล้ว ก่อนหน้าทั้งหมด
- **สถานะฐานข้อมูล:** ยอดเงินคงเหลือในบัญชี ID 3 ยังคงเป็น **1000.0** เท่าเดิม และไม่มีรายการฝากถูกเพิ่ม เข้าไปในตาราง deposit_transactions ทำให้ข้อมูลในฐานข้อมูลถูกต้องตามความเป็นจริง

2. กรณีไม่มี @Transactional (จากรูปภาพ image_ae844c.png และ image_ae844f.png)

- **พฤติกรรม:** แต่ละคำสั่ง SQL บันทึกลงฐานข้อมูลแยกจากกันทันที (Auto-commit)
- **ผลลัพธ์:** เมื่อขั้นตอนการฝากเงินทำงานถึงการอัปเดตยอดเงิน `accountRepository.save(account)` ข้อมูลยอดเงินจะถูกส่งไปบันทึกลงฐานข้อมูลทันที ทำให้ยอดเงินเปลี่ยนจาก 1000.0 กลายเป็น **2000.0** แต่เมื่อโปรแกรมรันมาถึงขั้นตอนถัดไปแล้วเกิด Error ทำให้กระบวนการหยุดลงโดยไม่สามารถย้อนคืน ยอดเงินที่เพิ่มไปแล้วได้
- **สถานะฐานข้อมูล:** เมื่อยิง GET /accounts/3 จะพบว่ายอดเงินเพิ่มขึ้นเป็น **2000.0** แล้ว แต่ประวัติการ ฝากในตาราง deposit_transactions (ลำดับที่ 7) อาจบันทึกไม่สมบูรณ์ หรือเกิดความขัดแย้งของข้อมูล ระหว่างสองตารางทันที

สรุปผลการทดลอง

การลบ @Transactional ออก ทำให้ระบบขาดคุณสมบัติ **Atomicity** ของฐานข้อมูล ส่งผลให้เมื่อเกิด ข้อผิดพลาด (Error) กลางคัน ระบบจะไม่สามารถ **Rollback** ย้อนกลับข้อมูลได้ ซึ่งสร้างความเสียหายร้ายแรงต่อ ระบบการเงิน (เช่น ยอดเงินถูกตัดหรือเพิ่มจริงแต่ไม่มีประวัติรายการ) ดังนั้นการใช้ @Transactional ในชั้น Service จึงมีความสำคัญอย่างยิ่งสำหรับกระบวนการทำงานที่มีหลายขั้นตอนต่อเนื่องกัน

คำถามท้าย Lab

1. @Transactional มีหน้าที่อะไร?

ทำหน้าที่กำหนดขอบเขตของการทำงานกับฐานข้อมูล (Database Transaction Boundary) ให้คำสั่ง SQL หลายๆ คำสั่งที่อยู่ในเมธอดนั้นนับเป็น "งานก้อนเดียวกัน" (Atomicity) เพื่อรับประกันความถูกต้องของข้อมูลตามหลัก ACID

2. เพราะเหตุใดการฝากเงินจึงควรใช้ Transaction?

เพราะการฝากเงิน 1 ครั้งประกอบด้วยหลายขั้นตอนที่ต้องทำต่อเนื่องกัน ได้แก่ **การอัปเดตยอดเงินคงเหลือ (balance) ในบัญชี** และ **การบันทึกประวัติการฝาก (DepositTransaction)** หากขั้นตอนใดขั้นตอนหนึ่งเกิดความผิดพลาด ทั้งสองขั้นตอนจะต้องถูกยกเลิกพร้อมกัน เพื่อป้องกันไม่ให้ข้อมูลขัดแย้งกัน (เช่น เงินเพิ่มแต่ไม่มีประวัติ หรือมีประวัติแต่เงินไม่เพิ่ม)

3. COMMIT และ ROLLBACK ต่างกันอย่างไร?

COMMIT: คือการยืนยันการบันทึกข้อมูลอย่างถาวรลงฐานข้อมูล เมื่อทุกขั้นตอนใน Transaction ทำงานสำเร็จสมบูรณ์โดยไม่มี Error

ROLLBACK: คือการยกเลิกและย้อนกลับการเปลี่ยนแปลงทั้งหมดที่เกิดขึ้นใน Transaction นั้น ให้ฐานข้อมูลกลับไปอยู่ในสภาพเดิมก่อนเริ่มทำการรายการ เมื่อเกิด Error หรือ Exception กลางทาง

4. Account และ DepositTransaction มีความสัมพันธ์แบบใด?

มีความสัมพันธ์แบบ **One-to-Many** (1 Account สามารถมีได้หลาย DepositTransaction) หรือมองในมุมของ DepositTransaction จะเป็น **Many-to-One** โดยอ้างอิงกลับไปยัง Account ผ่าน Foreign Key ชื่อ `account_id`

5. เมื่อเกิด Error ขณะใช้ @Transactional ข้อมูลใน Database เปลี่ยนแปลงอย่างไร?

ข้อมูลในฐานข้อมูล **จะไม่เปลี่ยนแปลงเลย (ไม่มีการบันทึกใดๆ เกิดขึ้น)** ระบบจะทำการ Rollback ย้อนคืนสถานะทั้งหมด เงินในบัญชีจะเท่าเดิม และไม่มีบันทึกประวัติรายการฝากเพิ่มขึ้นมา

6. จากการทดลอง เมื่อลบ @Transactional ออก ผลลัพธ์แตกต่างจากเดิมอย่างไร?

กรณีใช้ @Transactional (เดิม): หากเกิด Error ในขั้นตอนบันทึก DepositTransaction ยอดเงินที่ถูกเพิ่มไปใน Account ก่อนหน้านี้จะถูกสั่ง Rollback คืนกลับเป็นยอดเดิม

กรณีลบ @Transactional ออก: หากเกิด Error ในขั้นตอนบันทึก DepositTransaction ยอดเงินใน Account ที่ถูกอัปเดตไปแล้ว จะถูกบันทึกลงฐานข้อมูลทันทีและไม่ถูกยกเลิก ส่งผลให้เกิดปัญหาข้อมูลไม่สอดคล้องกัน (ยอดเงินเพิ่มขึ้นจริง แต่ประวัติการฝากบันทึกไม่สำเร็จ)